Chap: 3 22.10.2022

Binary search

compare the searching element with middle element.

Key element : mid

i/p { a : input data set
n : number of elements
x : searching element

let,
low = 1 high = 14

$$mid = \frac{\lfloor low + high \rfloor}{2} = \frac{\lfloor 1+14 \rfloor}{2} = 7.5 \simeq 7$$

★ How many comparisons are needed to know that the element is not present?

★ Is binary search algorithm correct or not?

Recursive binary search → divide & conquer approach

Return value stack → মূল্য ফেরত

when low > high → the whole array has been traversed but the element is not found,

If the whole tree is not traversed, then the algorithm is not correct. → Testing

Max Min

max = min = first element [এমন ধরে নিলে 1 টা comparison কমে যায়]

Improved: $\frac{n-1+2n-2}{2} = \frac{3n}{2} - \frac{3}{2}$

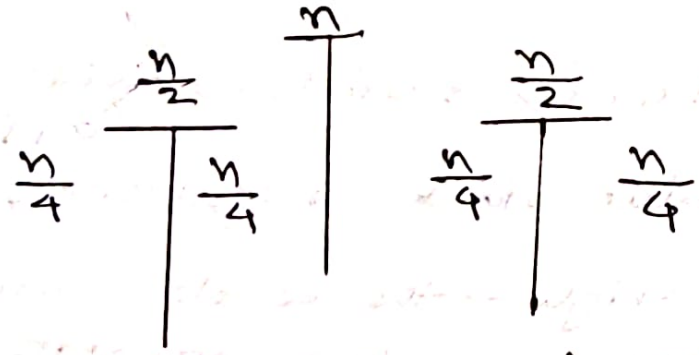
↓

as only 1 conditional statement is executed for best case. Hence, the comparison decreases by 1/2.

23.10.2022

Merge Sort

card ভাগার সময় use করা হয়।



till single element

Recursive

a : global array as it needs to be accessed from both function

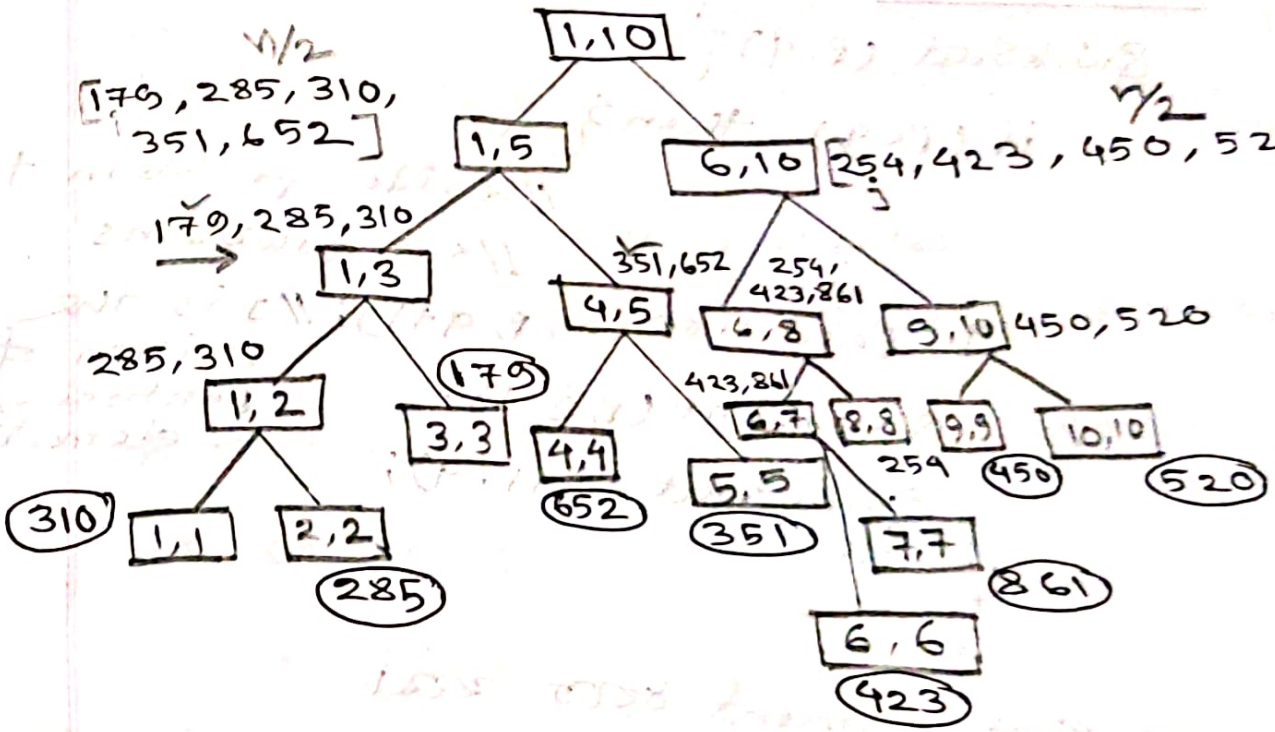
stack এ রাখতে অনেক জায়গা লাগে।

1	2	3	4	5	6	7	8	9	10
# 310	285	179	652	351	423	861	254	450	520

low = 1
high = 10

if (low < high) = T

n [179, 254, 285, 310, 351, 423, 450, 520, 652, 861]



swapping करने → merge function

$$T(n) = \begin{cases} a ; & n = 1 \rightarrow \text{as only merging operation has been done} \\ 2T(n/2) + cn ; & n > 1 \end{cases}$$

$n/2$ ↙ ↘ $n/2$ → 2 वार list को merge करने का time लगता है

Limitation

- space for merging the data
- stack space for recursion

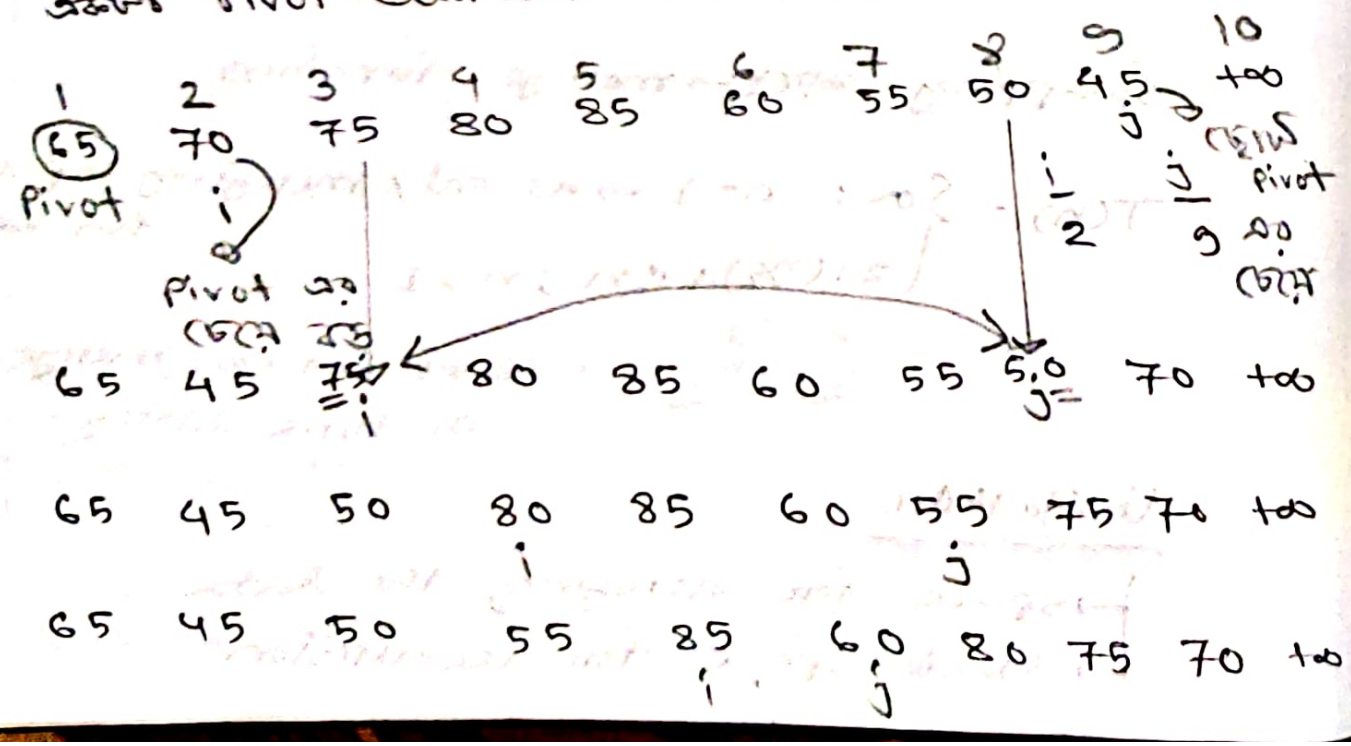
এটা solve করা সহজ, value না রেখে, address রাখলে। তাছাড়া, quicksort দিয়েও করা সম্ভব।

29.10.2022

Quicksort:

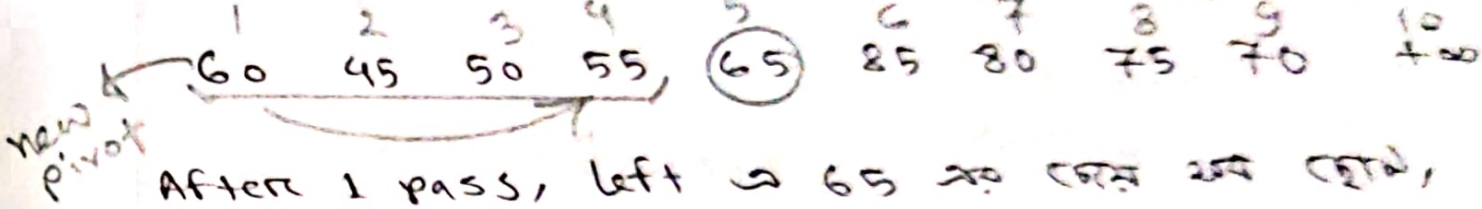
```
QuickSort (p, q){                    // p → 1st element, q → last element
    if (p<q) then{
        J = Partition(a, p, q+1);    // returns position // divide problem to sub problems
        QuickSort(p, j-1);           // j is the position of partitioning element
        QuickSort(j+1, q);
    }
```

এখানে pivot element ধরতে হবে।



65 45 50 55 60 85 80 75 70 +∞

i আর বাড়বে না, j আর কমবে না তখন, ith element এর সাথে Pivot element exchange করব।



After 1 pass, left এ 65 এর থেকে সব ছোট, right এ বড়। So, 5 এ partition হলো।

Quick_sort (1, 4)
Quick_sort (6, 9)
Pivot fix হওয়ার পর, আর নড়বে না। → 65

★ For sample data, how quicksort is applied?

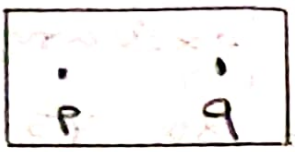
55 45 50 (60)

50 45 (55) (60)

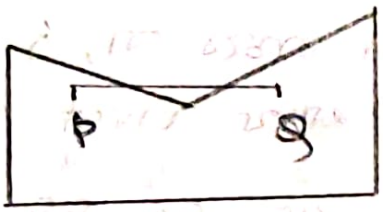
45 50 55 60 → sorted

Same way তে Right side ও sort হবে।

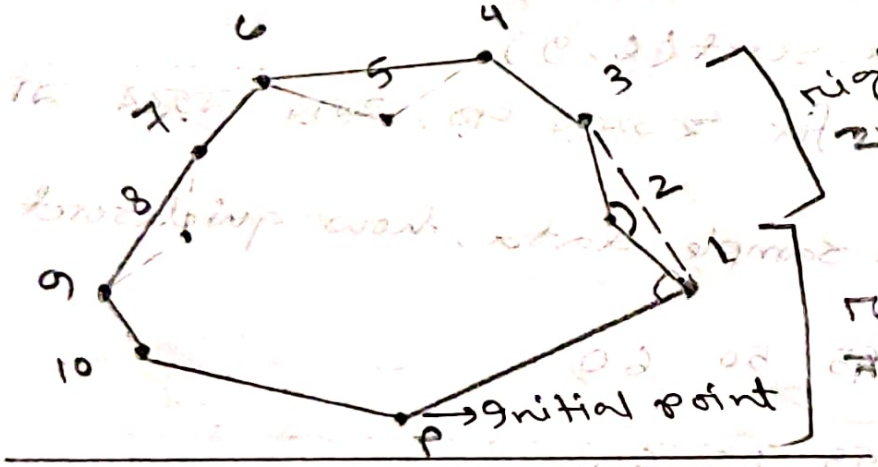
Convex Hull : The convex hull of a set of points s in the plane is defined as the smallest convex polygon containing all the points.



Convex



Not Convex



optimized Polygon

Convex polygon → দুটো point connect করলে, line টা সব polygon এর ভেতরে থাকে।

30.10.2022

Greedy Method:

Knapsack:

$n \rightarrow$ no. of items

$m \rightarrow$ total weight

$n = 3$

$m = 20$ [total weight]

$(P_1, P_2, P_3) = (25, 24, 15)$

$(w_1, w_2, w_3) = (18, 15, 10)$

$$\sum_i w_i x_i \leq m$$

$$\text{maximize} \sum_i P_i x_i$$

feasible solution এ কোন item কতটুকু নিচ্ছ তা বলতে হবে।

Profit = $P_i x_i$	x_1	x_2	x_3	weight = $w_i x_i$
12.5+8+3.75=24.25 random	1/2	1/3	1/4	= 9+5+2.5 = 16.5
25+3.2+0=28.2 more profit	1	2/15	0	= 18+2+0 = 20
0+16+15=31 less weight for more	0 item	10/15 = 2/3	1	= 0+10+10 = 20
0+24+7.5=31.5	0 more	1 Profit Per	5/unit = 1/2	= 0+15+5 = 20

Per unit profit, $1 \rightarrow \frac{25}{18} = 1.389$

$2 \rightarrow \frac{24}{15} = 1.6$

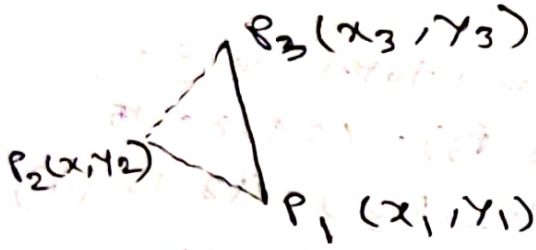
$3 \rightarrow \frac{15}{10} = 1.5$

[Algorithm এর ক্ষেত্রে problem সমাধান করি important]

05.11.2022

Convex Hull

Graham Scan



$\angle P_1 P_2 P_3$

$$D = \begin{vmatrix} x_1 & x_2 & x_3 \\ y_1 & y_2 & y_3 \\ 1 & 1 & 1 \end{vmatrix}$$

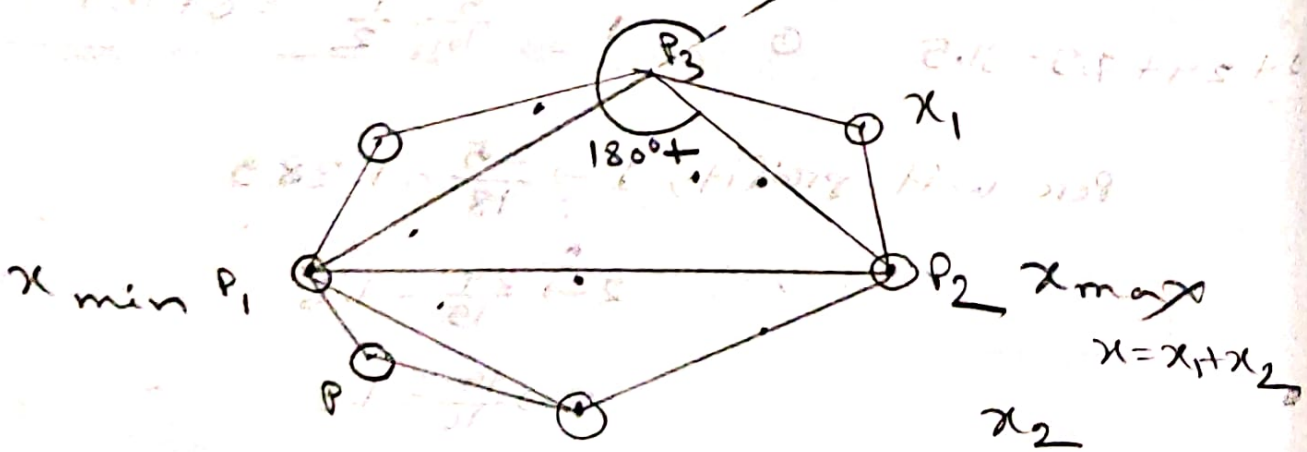
if $+D$, angle $< 180°$ left turn

if $-D$, angle $> 180°$ Right turn

if right turn, then we will exclude P_2.

Quick Hull Algorithm

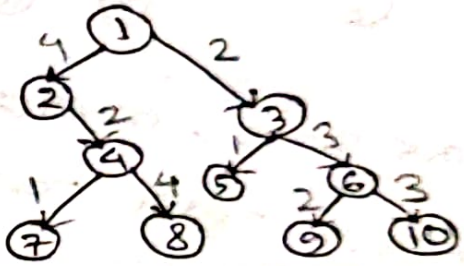
using divide & conquer approach



extreme point → largest x, smallest x
y- so
value

19.11.2022

Tree vertex splitting (TVS)



sink : 7, 8, 5, 9, 10

sink → loss/delay = 0 [as no other nodes are produced there]

$$d(u) = \max_{v \in c(u)} \{d(v) + w(u,v)\}$$

v = child

u = parent

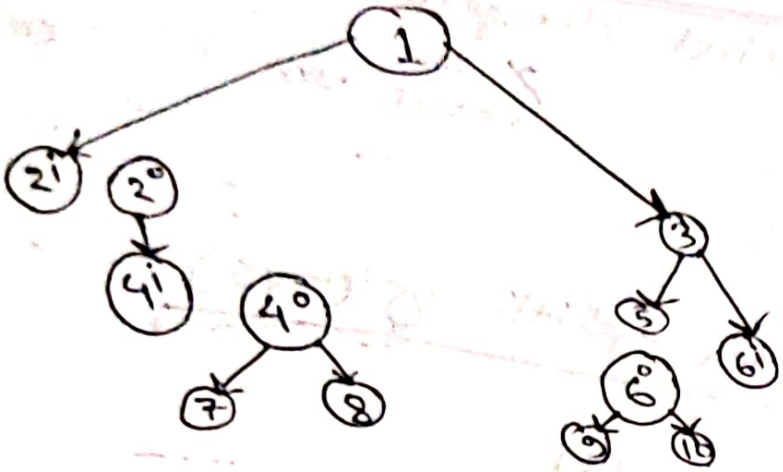
$$d(4) = \max\{(d(7) + d(7,4)), (d(8) + d(8,4))\}$$

$$= \max\{(0+1), (0+4)\}$$

$$= \max(1, 4) = 4$$

loss, $\delta = 5$ → highest

booster → signal regenerate



o = out
i = in

as 5 = 5, তোমার 5 > 5 হবে, তোমার boostem বসিয়ে split করতে হবে।

20.011.2022

Spanning Tree

Minimum spanning Tree (MST)
Kree ২৫

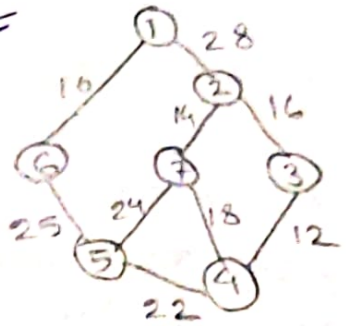
MST → No isolated vertex
connected graph → n−1 edge
n vertices

To create MST → Prim's Algo
→ Kruskal's Algo

Prim's Algo

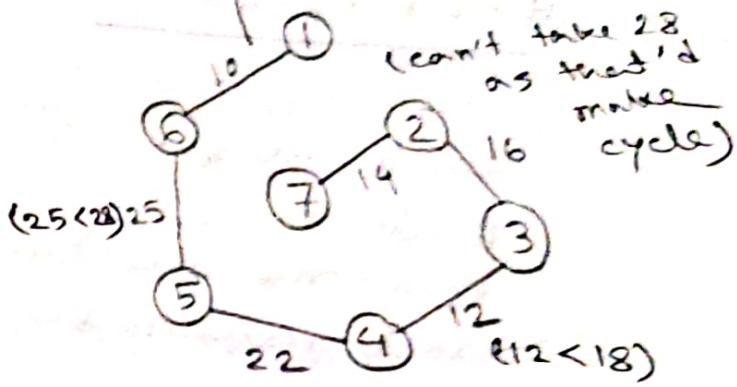
→ choose two vertex with minimum cost
→ next edge should be chosen as such that stays tree. For that there would be no cycle, and that has to be connected.

#



Graph

minimum weight (10)



MST

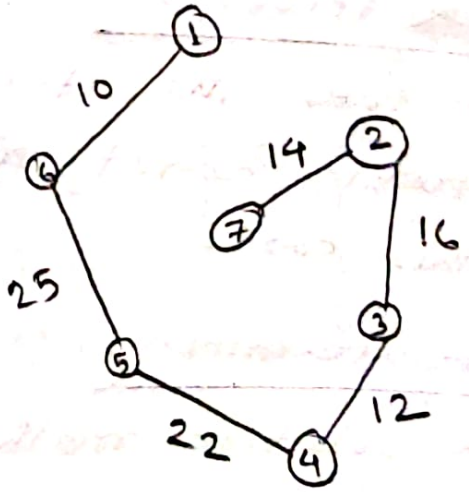
Total weight = 99

★ Local optima

→ If we choose local optima, we might not reach our goal.

Kruskal's algo

cycle না হলেই হলো। connected এর ব্যাপারে চিন্তা করতে হবে না। Forest হবে। প্রথমে edge sort করে নিতে হবে।



(4,7) → 18 X
[কারণ cycle হয়ে যাবে]

(1,6) 10
(3,4) 12
(2,7) 14
(2,3) 16
(4,7) X 18
(4,5) 22
(5,7) X 24
(5,6) 25
(1,2) X 28

26.11.2022

Backtracking

for larger search space
check all possible soln
Brute force
password checking, maze running
Exact solution by sacrificing time

n-Queens Problem ★

↳ Board Viva

কোনো soln পাওয়া সম্ভব না হলে, immediate আগের step এ backtrack করব।
যেখানে empty slot এর সংখ্যা কম থাকবে, সেখানে আগে বসানো → optimized.

★ how to place n-queens in a nxn chess board?

27.11.2022

0/1 Knapsack Problem

either we'll take the item or not.
Greedy approach can't give us optimal solution in this case.

Dynamic Programming.

— is a solution method that depends on the result of a sequence of decision.

– is a solution method that can be used when the solution to a problem can be designed as the result of a sequence of decision.

যদি কোনো solution কাজে না লাগে, তখন ঐ subset থেকে subset হিসেবে ধরা হয় না।

★ Recursive formula

# Item	I_1	I_2	I_3	I_4
Weight	2	3	4	5
Benefit	3	4	5	6

$w = 5$

finish

$w \rightarrow$

$i \downarrow$

B	0	1	2	3	4	5
0	0*	0	0	0	0	0
1	0	0	0+3	0+3	0+3	0+3
2	0	0	3	0+4	0+4	3+4
× 3	0	0	3	4	0+5	0+5
× 4	0	0	3	4	5	0+6

max in last (original) ← 2

[as last সব copy from previous]

→ 3−3=0

1st encounter

7−4=3

7

2 5 5<7

7

6<7

max profit

benefit table

item:	I_1	I_2	I_3	I_4
	1	1	×	×

optimal profit = 3+4=

$i = 4$

$w_i = 4$

$B[i,W] = \max(B[i-1,W], B[i-1,W-w_i] + b[i])$

$\Rightarrow B(4,4) = \max(B(3,4), B(3,5-5)+6)$

$= \max(5, 0+6) = 6$ → wrong

↑ it should be 5

! $B[4,4] = B(i-1,W)$ [as $w_i > W$]

$= B(4,4) =$

#Largest common string sequence finding ★

S_1: ABC BA

S_2: CBA

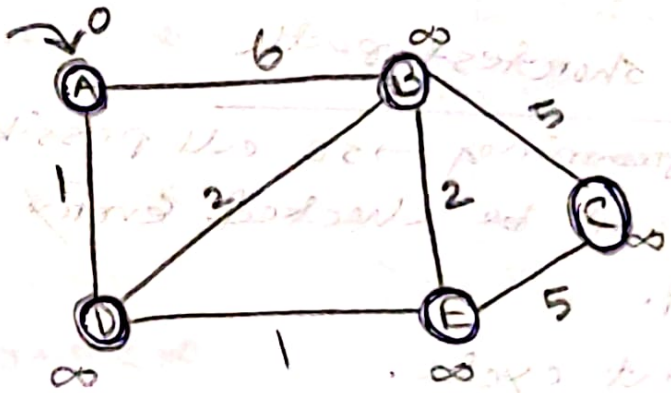
03.12.2022

Dijkstra's Algorithm

Single-Source Shortest Path

We will start from a single source. The starting point is very important.

Application → Mapping (map quest, Google maps)
Traffic Information System
Routing system



Solution: ADEBC

Vertex V	Shortest distance from A	Previous Vertex
A	0	
B	∞ 6 3	D
C	∞ 7	E
D	∞ 1	B
E	∞ 2	C

Relaxing Vertex

if $d(u) + w(u,v) < d(v)$

$d(v) = d(u) + w(u,v)$

i) $0+6<\infty \rightarrow A \rightarrow B$
$0+1<\infty \rightarrow A \rightarrow D$

ii) $1+1<\infty \rightarrow D \rightarrow E$
$1+2<6 \rightarrow D \rightarrow B$

iii) $2+5<\infty \rightarrow E \rightarrow C$

A → B : A → D → B

A → C : A → D → E → C

A → D : A → D

A → E : A → D → E

Bellman ford

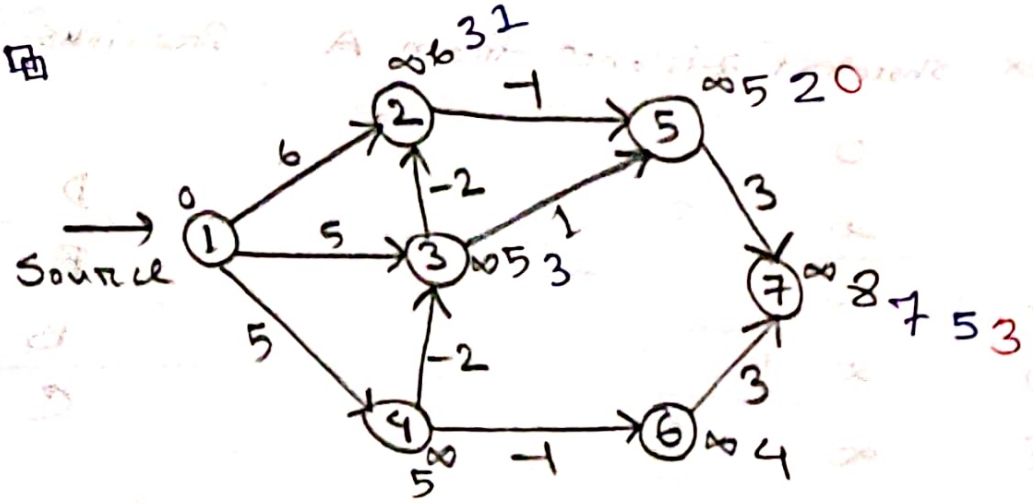
Con of Dijkstra → can't take negative weight.

★ Negative weight cycle solution

Single-source shortest path

dynamic programming → so all possible combination will be checked. Every state will be saved.

negative weight cycle.



No. of vertex, n

the algorithm will run (n-1) times max.

(n-1) বার আগেই যদি সব বার হয় যে weight আর update হচ্ছে না, তাহলে stable.

Possible Edges: (1,2), (1,3), (1,4), (2,5), (3,2), (3,5), (4,3), (4,6), (5,7), (6,7)

Vertex: 7

∴ The algorithm will run maximum 6 times

If $d(u) + w(u,v) < d(v)$
then $d(v) = d(u) + w(u,v)$ } Relaxation

1st i) (1,2)
$0+6 < \infty$

ii) (1,3)
$0+5 < \infty$

iii) (1,4)
$0+5 < \infty$

iv) (2,5)
$6-1 < \infty$

v) (3,2)
$5-2 < \infty$

vi) (3,5)

vii) (4,3)
$5-2$

viii) (4,6)
$5-1$

ix) (5,7)
$5+3$

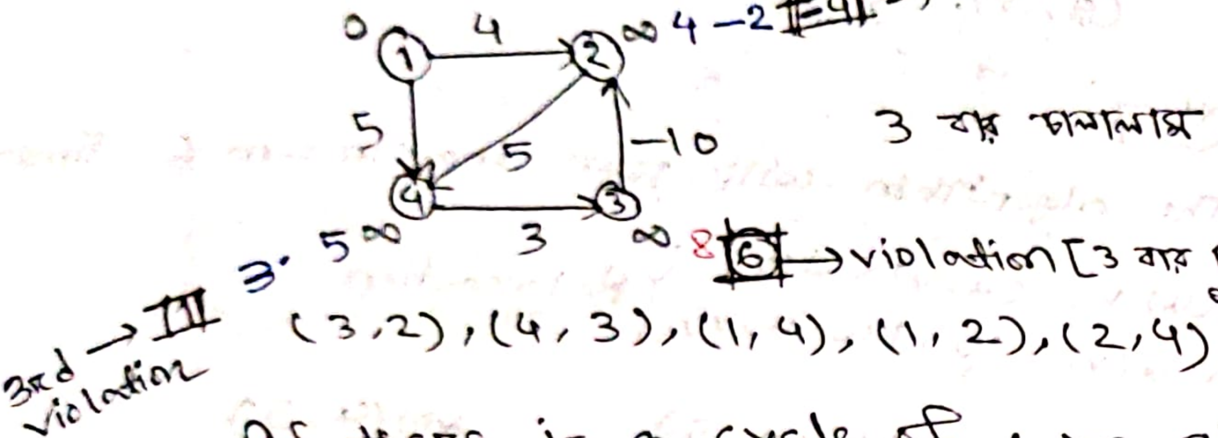
x) (6,7)
$4+3$

এভাবে আরো দুইবার loop চলবে।

1 — 0
2 — 1
3 — 3
4 — 5
5 — 0
6 — 4
7 — 3

} distance from first node to all the nodes.

Limitation



3 বার চালালাম

[3 বার pass করার পরও update হচ্ছে]

(3,2), (4,3), (1,4), (1,2), (2,4)

If there is a cycle of (−)ve element (negative cycle), then Bellman Ford can't solve it.

Violations are happening due to (−)ve cycle.

04.12.2022

Travelling Salesman Problem

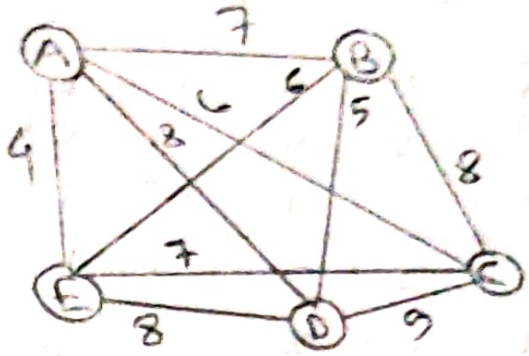
to minimize travelling distance.

Bellman ford এ starting point ফিক্সড করতে হবে which is problematic.

TSP তে যেকোনো জায়গায় শুরু করে, minimum distance use করে সব cover করতে হবে।

No. of node & edge বাড়লে, TSP polynomial time এ solve করতে হয়।

#



Matrix solution

	A	B	C	D	E
A	—	7	6	8	4
B	7	—	8	5	6
C	6	8	—	9	7
D	8	5	9	—	8
E	4	6	7	8	—

1. Row minimization

Row scan করে, ঐ row এর সবচেয়ে ছোট value, row এর বাকি element থেকে বিয়োগ করব।

	A	B	C	D	E
A	—	3	2	4	0
B	2	—	3	0	1
C	0	2	—	3	1
D	3	0	4	—	3
E	0	2	3	4	—

2. Column minimization

	A	B	C	D	E
A	–	3	0^1	4	0^1
B	2	–	1	0^4	1
C	0^1	2	–	3	1
D	3	0^4	2	–	3
E	0^1	2	1	4	–

corner ও penalty নেওয়া।

3. Penalty calculation

	A	B	C	D	E
A	–	3	1	4	1
B	2	–	1	(4)	1
C	1	2	–	3	1
D	3	4	2	–	3
E	1	2	1	4	–

$\therefore B \rightarrow D$

for $0 \rightarrow$ যে 0 যে row ও আছে, আর যে column ও আছে, সেই দুই row & column এর সবচেয়ে ছোট number (0 বাদে) দুটোকে যোগফল 0 এর পাশায় বসবে।

সর্বোচ্চ penalty এর row & column বাদ দিয়ে দিব। আর দুটো বা এর বেশি same সর্বোচ্চ

number থাকলে, সেক্ষেত্রে সবসময় মিলেই হবে।

	A	B	C	E
A	–	3	D	0
C	0	2	–	1
D	3	~~0~~ –	2	3
E	0	2	1	–

As B→D or D→B হয়ে গেছে, তাই আর calculation এ আর যাব না।
Again repeat all the steps.

after row–col minimization

	A	B	C	E
A	–	1	0 0	0 1
C	0 0	0 0	–	1
D	1	–	0 1	1
E	0 0	0 0	1	–

	A	B	C	E
A	–	1	0	(1)
C	0	0	–	1
D	1	–	(1)	1
E	0	0	1	–

∴ A→E

	A	B	C
C	0^1	0^0	—
D	1	—	0^2
E	—	0^1	1

No minimization needed as all the rows & cols contain 0.

	A	B	C
C	1	0	—
D	1	—	2
E	—	1	1

$\therefore D \rightarrow C$

	A	B
C	0^0	0^0
E	—	0^0

$\therefore E \rightarrow B$

As $A \rightarrow E$ already exists, B আর E বাদ দিলাম। $A \rightarrow E$ নিলে, solution double হয়ে যেত।

Hence, the whole solution,

1. $B \rightarrow D$
2. $A \rightarrow E$
3. $D \rightarrow C$
4. $E \rightarrow B$
5. $C \rightarrow A$

$\therefore B \rightarrow D \rightarrow C \rightarrow A \rightarrow E \rightarrow B$

Algorithm { limitation
complexity
benefit
which algo from multiple options

CT-03

Dijkstra, Bellman ford, Travelling Salesman

17.12.2022

P, NP, NP-complete, NP-Hard → class of problems

Big Oh → upper bound

Omega → lower bound

★ Def. of notation & calculation of notation.

P → polynomial time

$O(n^k)$, n → size of input

NP → Non-Deterministic Polynomial

★ P is a subset of NP.

★ Difference between NP, P, NP-hard!

★ Definition & types of problems of classes.

24.12.2022

Bin Packing (1-D)

No optimal solution

Next-fit → ১ম memory slot fill করে, তারপর ২য় টা fill করে --- পরের slot এ আসলে পর, previous slot এ আর back করা যাবে না।

First-fit → 1st slot আগে fill করে নিতে হবে। পরের slot এ গেলেও, back এ এসে fill করা যাবে।

Optimal → Available job sort করতে হবে প্রথমে (বড় → ছোট), size অনুসারে fit করব